Reinforcement Learning

Lecture 3 — Dynamic Programming

Przemysław Witaszczyk

Faculty of Physics, Astronomy and Applied Computer Science
Jagiellonian University

2026/2027

Where we are

Lecture 2 *characterized* the solution of an MDP: the four Bellman equations. They characterize what v_π, q_π, v_*, q_* satisfy — not how to *compute* them.

This lecture turns the equations for v_π and v_* into *algorithms*. The single conceptual move:

a Bellman equation is a **fixed-point condition**; *turn it into an* **assignment** *and iterate.*

$$v = \mathcal{B}\, v \qquad \longrightarrow \qquad v_{k+1} \leftarrow \mathcal{B}\, v_k .$$

Two Bellman equations give two operators, hence two algorithm families:

Bellman equation (L2)	$\longrightarrow$	**iterate the operator (this lecture)**
expectation $v_\pi = \mathcal{B}_\pi v_\pi$ (3.14)	$\longrightarrow$	$v_{k+1} \leftarrow \mathcal{B}_\pi v_k$ (policy evaluation)
optimality $v_* = \mathcal{B}_* v_*$ (3.19)	$\longrightarrow$	$v_{k+1} \leftarrow \mathcal{B}_* v_k$ (value iteration)

Assumption in this lecture: the dynamics $p(s', r \mid s, a)$ are **fully known**. This is *planning*. Lecture 4 drops the model.

Dynamic programming and the perfect-model assumption

Dynamic Programming (DP) is a family of algorithms that compute optimal policies *given a perfect model of the MDP*.

Value functions play two roles, both established in L2:

1. they **organize the search** for good policies, and
2. the **Bellman equations are the update rules** — read right-to-left, they are assignments.

Two reasons to study DP despite the model requirement and full sweeps:

- It is the **theoretical foundation** of the methods developed in the rest of the course.
- Almost every later method is "DP with the model replaced by *samples*": Monte Carlo, TD, SARSA, Q-learning, actor–critic all reproduce the DP steps with sampled updates.

The two Bellman operators

Define, on the space of value functions $v : \mathcal{S} \to \mathbb{R}$, the **Bellman expectation operator** $\mathcal{B}_\pi$ and the **Bellman optimality operator** $\mathcal{B}_*$:

$$(\mathcal{B}_\pi v)(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \big[r + \gamma v(s') \big],$$

$$(\mathcal{B}_* v)(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \big[r + \gamma v(s') \big].$$

These are L2's equations (3.14)/(3.19) with v on the right made a *generic* input. By the Bellman equations of L2,

$$v_\pi = \mathcal{B}_\pi v_\pi, \qquad v_* = \mathcal{B}_* v_*$$

— i.e. v_π, v_* are the **fixed points** of these operators. One pattern governs the lecture: *an operator is iterated to its fixed point, and the policy is then improved.*

Iterative policy evaluation

Problem (prediction): given a fixed policy π, compute v_π.

Idea: start from any v_0 (terminals $= 0$) and repeatedly apply $\mathcal{B}_\pi$:

$$v_{k+1}(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \big[r + \gamma v_k(s') \big]. \qquad (4.5)$$

The sequence $\{v_k\}$ converges to v_π (proved by the contraction argument below).

This is an **expected (full) update**: it uses the *entire* distribution p over (s', r). Monte Carlo (L4) and TD (L5) will replace this exact expectation by *sampled estimates* (returns for MC, transitions for TD).

The linear-system view of policy evaluation

For a *fixed* π the Bellman expectation equation is **linear** in the unknowns $\{v_\pi(s)\}$: it is $|\mathcal{S}|$ equations in $|\mathcal{S}|$ unknowns. In matrix form, with $r_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p\, r$ and transition matrix $(P_\pi)_{s,s'} = \sum_a \pi(a|s) \sum_r p(s', r \mid s, a)$,

$$v_\pi = r_\pi + \gamma P_\pi v_\pi \qquad \Longrightarrow \qquad v_\pi = (I - \gamma P_\pi)^{-1} r_\pi .$$

For $\gamma < 1$ it is always well-posed. P_π is stochastic (rows sum to 1), so γP_π has spectral radius $\leq \gamma < 1$; hence 1 is never an eigenvalue, $(I - \gamma P_\pi)$ is invertible, and v_π *exists and is unique*. This is the same γ-contraction, now read as a spectral condition.

The reason for iterating: the direct solve is an $O(|\mathcal{S}|^3)$ matrix inverse — infeasible once $|\mathcal{S}|$ is large (the curse of dimensionality again). Iteration replaces the exact solve with repeated sweeps of lower per-sweep cost.

The reason for writing it down: this matrix is where an **instability of Lecture 9 originates**: there, once values are *approximated* (shared weights instead of a table) and training is *off-policy*, the analogue of $(I - \gamma P_\pi)$ — now weighted by the *behavior* data distribution — can lose positive definiteness, and the iteration *diverges*: the **deadly triad** (approximation + bootstrapping + off-policy). Tabular DP here is the stable baseline against which that instability is defined.

The algorithm box

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π, the policy to be evaluated
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\mathit{terminal})$ to 0

Loop:
  $\Delta \leftarrow 0$
  Loop for each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\left[r + \gamma V(s')\right]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Two-array vs in-place. The box is *in-place*: one array, overwriting $V(s)$ as the sweep proceeds. The two-array version keeps v_k and v_{k+1} separate. In-place uses fresh values within a sweep and usually converges *faster*; the sweep order then matters — the starting point of **asynchronous DP** (treated below).

Stopping rule. Track $\Delta = \max_s |v_{k+1}(s) - v_k(s)|$ and stop when $\Delta < \theta$.

(Sutton & Barto, Ch. 4.1)

Worked numbers: bootstrapping on Slippery Walk Five

SWF (Morales): states $0-6$, 0 and 6 terminal, reward $+1$ on reaching goal 6, $\gamma = 1$. The slip model: each move goes as intended w.p. $\frac{1}{2}$, stays put w.p. $\frac{1}{3}$, slips *backward* w.p. $\frac{1}{6}$. Evaluate the *always-left* policy — a policy that moves away from the goal, yet whose slips occasionally move the agent toward it. One sweep updates every state via (4.5); e.g. the first nonzero value, at state 5:

$$v_1(5) = \underbrace{\tfrac{1}{2}}_{\to 4}(0{+}0) + \underbrace{\tfrac{1}{3}}_{\to 5}(0{+}0) + \underbrace{\tfrac{1}{6}}_{\to 6}(1{+}0) = \tfrac{1}{6} \approx 0.1667.$$

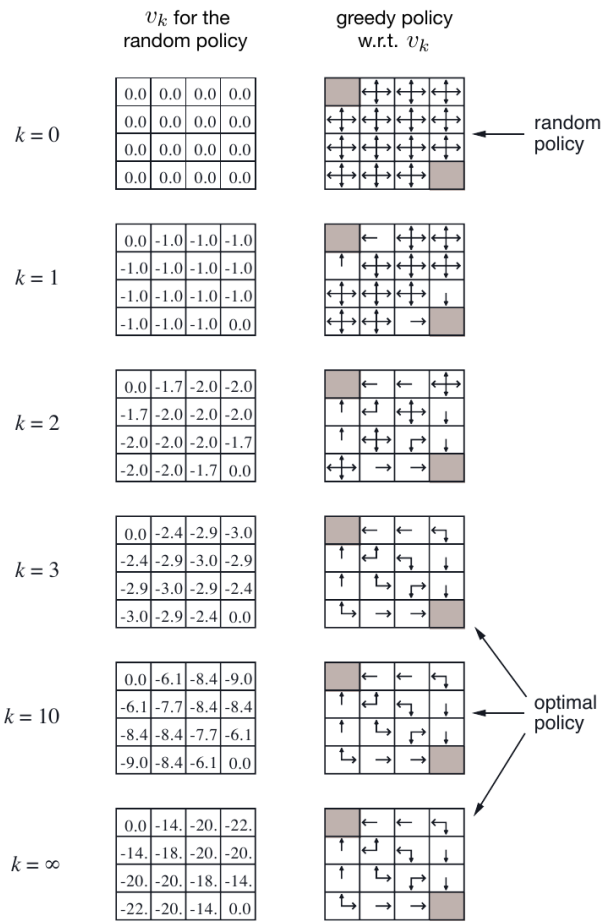
k	$V(1)$	$V(2)$	$V(3)$	$V(4)$	$V(5)$
1	0	0	0	0	0.1667
2	0	0	0	0.0278	0.2222
3	0	0	0.0046	0.0463	0.2546
5	0.0001	0.0018	0.0135	0.0705	0.2883
10	0.0014	0.0067	0.0267	0.0959	0.3180
∞	0.0027	0.0110	0.0357	0.1099	0.3324

Bootstrapping: each v_{k+1} is computed from the *previous estimate* v_k — "an estimate from an estimate."

Value propagates **outward from the goal**: state 5 first, then $4, 3, \ldots$ A sweep of (4.5) is one row; the table shows the convergence (absorption; $\gamma = 1$).

The convergence argument below assumes $\gamma < 1$, whereas SWF sets $\gamma = 1$. Sutton's condition (§4.1) is $\gamma < 1$ *or* eventual termination from all states under π — SWF terminates w.p. 1 under any policy, so the result holds, over several sweeps (absorption plays the rôle of γ). (Morales, Ch. 3 — Slippery Walk Five)

Worked trace: the 4×4 small gridworld



Convergence of iterative policy evaluation on a small gridworld. The left column is
of approximations of the state-value function for the random policy (all actions
). The right column is the sequence of greedy policies corresponding to the value
mates (arrows are shown for all actions achieving the maximum, and the numbers
ounded to two significant digits). The last policy is guaranteed only to be an
over the random policy, but in this case it, and all policies after the third iteration,

Equiprobable random policy; reward -1 per step until a terminal corner.

Left: v_k for $k = 0, 1, 2, 3, 10, \infty$ — the values approach v_π sweep by sweep.
Right: the policy *greedy* w.r.t. each v_k.

The key observation: the greedy policy is already **optimal at** $k = 3$, long before v_k converges ($k = \infty$).

Hence evaluating *to convergence* before improving does more work than improvement needs. This directly motivates **truncation** (value iteration) and the looseness of **GPI**.

Convergence of policy evaluation (the proof deferred in L2)

Claim. $\mathcal{B}_\pi$ is a γ-**contraction in the sup-norm** $\|u\|_\infty = \max_s |u(s)|$:

$$\|\mathcal{B}_\pi u - \mathcal{B}_\pi v\|_\infty \;\le\; \gamma\, \|u - v\|_\infty.$$

Proof. For each s, the policy-and-dynamics weights $w(a, s', r) \doteq \pi(a \mid s)\, p(s', r \mid s, a)$ are nonnegative and sum to 1 over (a, s', r) — a *convex average*. Subtracting $\mathcal{B}_\pi u$ and $\mathcal{B}_\pi v$, the rewards r are identical in both and cancel, leaving only the $\gamma\, v(s')$ terms:

$$\begin{aligned} |(\mathcal{B}_\pi u)(s) - (\mathcal{B}_\pi v)(s)| &= \gamma \Big| \textstyle\sum_{a,s',r} w(a, s', r) \big[u(s') - v(s')\big] \Big| \\ &\le \gamma \textstyle\sum_{a,s',r} w(a, s', r) \big|u(s') - v(s')\big| \\ &\le \gamma \max_{s'} |u(s') - v(s')| = \gamma\, \|u - v\|_\infty. \end{aligned}$$

Line 2 is the triangle inequality ($w \ge 0$); line 3 is the key fact: a weighted average cannot exceed the largest magnitude it averages, since $\sum_i w_i |x_i| \le (\sum_i w_i) \max_i |x_i| = \max_i |x_i|$ for convex weights (they sum to 1). Taking $\max_s$ on the left gives the claim. $\square$

What the contraction yields: existence, uniqueness, convergence rate

Consequence (Banach fixed-point theorem). On the complete space $(\mathbb{R}^{|\mathcal{S}|}, \|\cdot\|_\infty)$, a γ-contraction with $\gamma < 1$ gives:

- a **unique** fixed point exists — and since v_π satisfies $\mathcal{B}_\pi v_\pi = v_\pi$ (the Bellman equation), that fixed point *is* v_π (the existence/uniqueness deferred from L2);
- iteration converges **geometrically from any start**: $\|v_k - v_\pi\|_\infty \leq \gamma^k \|v_0 - v_\pi\|_\infty$.

Repeated backups do more than improve the estimate: they *must* converge to v_π, at a known rate. Concretely, at $\gamma = 0.9$ each sweep shrinks the worst-case error by 10%, so $\sim$22 sweeps shrink the worst-case error by a further factor of ten ($0.9^{22} \approx 0.098$).

The advantage function: gain from a one-step deviation

From v_π and q_π Morales derives a third value function, the **action-advantage function** (":=" means "is defined as"):

$$A_\pi(s,a) \;:=\; q_\pi(s,a) - v_\pi(s).$$

It measures how much better action a is than the policy's *default* behaviour in s:

- $A_\pi(s,a) > 0$: a improves on π's behaviour at s;
- **it is zero on π-average** — averaging the definition over $a \sim \pi(\cdot \mid s)$ and using $\sum_a \pi(a \mid s) q_\pi(s,a) = v_\pi(s)$ (L2):

$$\sum_a \pi(a \mid s) A_\pi(s,a) = \underbrace{\sum_a \pi(a \mid s) q_\pi(s,a)}_{= v_\pi(s)} - v_\pi(s) = 0,$$

so an average action has no advantage — only *better-than-average* ones do. Improving a policy therefore means **picking the highest-advantage action** — exactly the greedy step $\arg\max_a q_\pi(s,a)$ developed below (the $-v_\pi(s)$ is constant in a).

Forward reference: the advantage is central to Lecture 11's actor–critic methods.

The greedy policy and the one-step improvement condition

Given v_π, form the **greedy policy** w.r.t. it:

$$\pi'(s) \;=\; \arg\max_a q_\pi(s,a) \;=\; \arg\max_a \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma v_\pi(s')\big].$$

At each state π' selects the action of maximal value under *one step* of the true dynamics, with v_π thereafter.

By construction it satisfies, for every s,

$$q_\pi\big(s,\pi'(s)\big) \;=\; \max_a q_\pi(s,a) \;\geq\; v_\pi(s).$$

That this one-step condition yields an at-least-as-good policy overall is exactly what the policy-improvement theorem guarantees.

Policy improvement theorem — statement

Theorem. Let π, π' be any pair of deterministic policies such that

$$q_\pi\big(s, \pi'(s)\big) \;\geq\; v_\pi(s) \qquad \text{for all } s \in \mathcal{S}. \tag{4.7}$$

Then π' is at least as good as π:

$$v_{\pi'}(s) \;\geq\; v_\pi(s) \qquad \text{for all } s \in \mathcal{S}.$$

Moreover, if the inequality (4.7) is strict at any state, then $v_{\pi'} > v_\pi$ there (strictly better policy).

The **greedy** π' defined above satisfies the hypothesis with $q_\pi(s, \pi'(s)) = \max_a q_\pi(s, a) \geq v_\pi(s)$. So greedy improvement never decreases v, and increases it unless π is already optimal.

Policy improvement theorem — proof

Start from the hypothesis and **unroll**, re-applying it to the tail each time (the tower property from L1/L2 collapses each conditional expectation):

$$
\begin{aligned}
v_\pi(s) &\le q_\pi\big(s, \pi'(s)\big) = \mathbb{E}_{\pi'}\big[R_{t+1} + \gamma\, v_\pi(S_{t+1}) \,|\, S_t{=}s\big] \\
&\le \mathbb{E}_{\pi'}\big[R_{t+1} + \gamma\, q_\pi(S_{t+1}, \pi'(S_{t+1})) \,|\, S_t{=}s\big] \\
&= \mathbb{E}_{\pi'}\big[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \,|\, S_t{=}s\big] \\
&\;\;\vdots \\
&\le \mathbb{E}_{\pi'}\big[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \,|\, S_t{=}s\big] = v_{\pi'}(s). \qquad \square
\end{aligned}
$$

Line by line: (1) def. of q_π, and taking $A_t{=}\pi'(s)$ *is* acting under π' for one step; (2) apply the hypothesis (4.7) at S_{t+1}; (3) expand that q_π the same way; then repeat. The step repeated each round: replace a v_π by $q_\pi(\cdot, \pi'(\cdot)) \ge v_\pi$, then unfold *that* q_π into "one more real π'-reward $+\gamma\, v_\pi$ of the next step". The v_π term is pushed ever further out ($\gamma^k \to 0$), leaving the full return under π' — i.e. $v_{\pi'}(s)$.

Policy improvement theorem — why PI stops at the optimum

Corollary. Suppose policy iteration reaches a π' that is greedy w.r.t. v_π yet *no better* — $v_{\pi'} = v_\pi$. Because π' is greedy,

$$v_\pi(s) = v_{\pi'}(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v_\pi(s')\big] \quad \text{for all } s.$$

But that is exactly the Bellman *optimality* equation (L2, eq. 3.19) — and its solution is unique. Hence $v_\pi = v_*$, and $\pi = \pi' = \pi_*$.

The consequence: improvement can stall only *at the optimum*. As long as π is suboptimal, greedy improvement produces a *strictly* better policy — so policy iteration cannot terminate short of π_*. This corollary underlies every value-based control method in the course.

Policy iteration

Alternate the two steps until the policy stops changing:

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_*$$

- **E** **evaluate**: compute v_{π_k} (iterative policy evaluation);
- **I** **improve**: $\pi_{k+1} = \text{greedy}(v_{\pi_k})$.

Each π_{k+1} is *strictly* better than π_k unless already optimal (improvement theorem). A **finite** MDP has only finitely many deterministic policies, so the strictly-improving chain must **terminate** — at an *optimal* policy, by the corollary.

Policy iteration — the algorithm box

ıction changes little from one policy to the next).

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

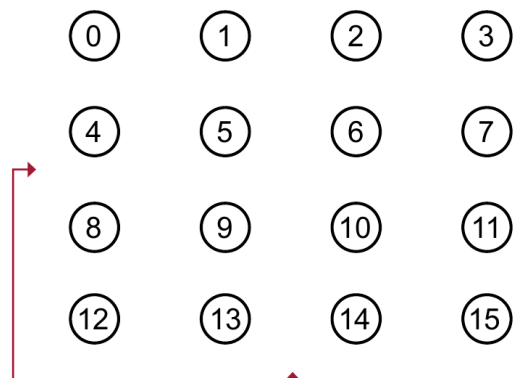
1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\mathit{terminal}) \doteq 0$

2. Policy Evaluation
   Loop:
   $\Delta \leftarrow 0$
   Loop for each $s \in \mathcal{S}$:
   $v \leftarrow V(s)$
   $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s))\big[r + \gamma V(s')\big]$
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   old-action $\leftarrow \pi(s)$
   $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
   If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

(Sutton & Barto, Ch. 4.3)

Worked trace: policy iteration on Frozen Lake



Frozen Lake (Morales): start `S`, holes `H`, goal `G`; slippery transitions; $r = 1$ only on reaching `G`.

One full round on the environment from L2, starting from the uniform-**random** π_0 ($\gamma = 0.99$), proceeds as follows:

1. **Evaluate** $\pi_0 \to v_{\pi_0}$ (sweep eq. 4.5 to $\Delta < \theta$):

.012	.010	.019	.009
.015	H	.039	H
.033	.084	.138	H
H	.170	**.434**	G

2. **Improve**, cell by cell — e.g. at state 14 (left of `G`): $q(14, \cdot) =$ (L .245, D .**533**, R .522, U .435) so the improved action is **D**, the slip-robust choice (never slides off the bottom row; still reaches `G` w.p. $\frac{1}{3}$).

Repeat. Within a few rounds the arrows stop changing (*policy-stable*), at which point $\pi_k = \pi_*$.

Value iteration

Policy iteration evaluates *to convergence* before each improvement — but the gridworld trace showed the greedy policy stabilizes early. **Value iteration** collapses evaluation to a *single sweep that already contains the max*:

$$v_{k+1}(s) \;\leftarrow\; \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_k(s')\big] \;=\; (\mathcal{B}_* v_k)(s). \tag{4.10}$$

— the Bellman *optimality* equation turned into an update.

Convergence: $\mathcal{B}_*$ is **also a γ-contraction** in $\|\cdot\|_\infty$. The one new ingredient is that $\max_a$ is **non-expansive**, $\big|\max_a f(a) - \max_a g(a)\big| \le \max_a |f(a) - g(a)|$; with that, the $\mathcal{B}_\pi$ proof goes through *unchanged in structure* (replace $\sum_a \pi(a \mid s)$ by $\max_a$). Hence the same **geometric error bound**:

$$\|v_k - v_*\|_\infty \;\le\; \gamma^k \, \|v_0 - v_*\|_\infty.$$

Extract the policy once at the end: $\pi_* = \text{greedy}(v_*)$.

Value iteration — the algorithm box

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $\quad v \leftarrow V(s)$
| $\quad V(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)\big[r + \gamma V(s')\big]$
| $\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r|s, a)\big[r + \gamma V(s')\big]$

(Sutton & Barto, Ch. 4.4)

Value iteration = truncated policy iteration

The two control algorithms are **endpoints of one spectrum**, parameterized by the amount of evaluation performed between improvements.

	evaluation per improvement	
Policy iteration	sweep to convergence (v_{π_k})	accurate, costly steps
Value iteration	exactly *one* sweep (with the max)	low-cost steps, more of them
In between	a few sweeps (*modified* PI)	tune the trade-off

Modified policy iteration is the named middle ground: run a *fixed* number m of evaluation sweeps between improvements. $m = \infty$ recovers policy iteration; $m = 1$ recovers value iteration. It often converges faster than VI — and it makes precise that PI and VI are not two algorithms but two settings of the single parameter m.

Both interleave **evaluation** and **improvement**; they differ only in the granularity. That looseness is exactly what **Generalized Policy Iteration** formalizes, once the sweep order is loosened as well (below).

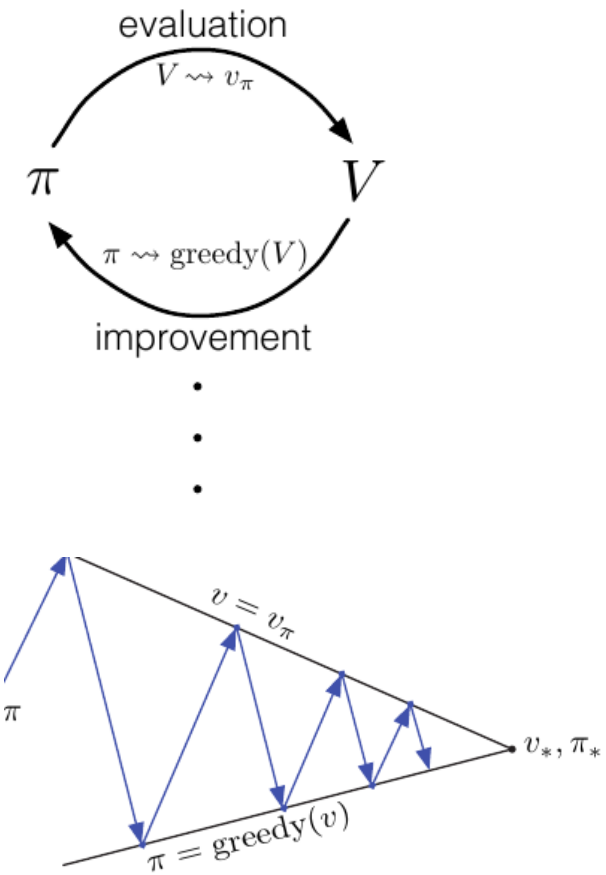
Asynchronous DP

A *sweep* updates every state once, in lockstep. This is not required.

Asynchronous DP updates states *in any order*, possibly repeating some and skipping others within a "sweep" — and, for $\gamma < 1$, still converges to v_* *provided every state continues to be updated* (visited infinitely often in the limit). The **in-place** sweeps of the evaluation and value-iteration boxes are the simplest special cases; this is why they stay correct despite mixing old and new values.

- Computation can be **focused** where values are changing most (cf. prioritized sweeping).
- Planning can be **interleaved with real-time interaction**, updating the states the agent visits. (Lecture 7 develops this as **Dyna** and real-time DP.)

Generalized Policy Iteration (GPI)



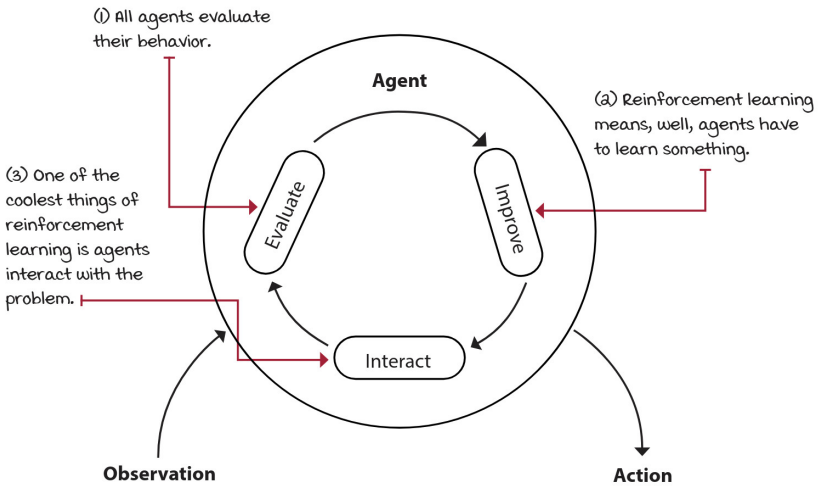
GPI is the general scheme in which *evaluation* and *improvement* interact, regardless of granularity.

- **Evaluation** makes v consistent with π $(v \rightsquigarrow v_\pi)$.
- **Improvement** makes π greedy w.r.t. v $(\pi \rightsquigarrow \text{greedy}(v))$.

The two pull toward the *same* joint fixed point (v_*, π_*): they compete locally (each makes the other "wrong"), but cooperate globally (lower diagram — the two constraints meet only at the optimum).

Almost every algorithm in this course is an instance of GPI. The same diagram covers PI, VI, MC control, SARSA, Q-learning, and actor–critic: each is "evaluate $\leftrightarrow$ improve" with a different evaluation rule.

GPI as the loop introduced in Lecture 1



The agent's **Evaluate** $\rightarrow$ **Improve** $\rightarrow$ **Interact** loop was introduced in Lecture 1 (the bandit's inner cycle). Each stage now has a name:

- **Evaluate** = policy evaluation (estimate the value of current behaviour),
- **Improve** = act greedily w.r.t. that estimate,
- **Interact** = (from L4) gather the data that makes evaluation possible *without* a model.

The bandit case has one state; DP uses a full model; the methods that follow use *samples*. (Morales, Ch. 2–3)

Efficiency of DP & the curse of dimensionality

The upper bound. To find an optimal policy DP takes time *polynomial* in $|\mathcal{S}|$ and $|\mathcal{A}|$ — exponentially faster than brute-force search over the $|\mathcal{A}|^{|\mathcal{S}|}$ deterministic policies. DP is, in this sense, efficient and is the practical method for moderately large MDPs.

The obstacle — the curse of dimensionality, inherent in the problem, not in DP (Sutton §4.7). $|\mathcal{S}|$ itself grows *exponentially* in the number of state variables. A model with d features, n values each, has n^d states — sweeping them all is infeasible.

Two responses organize the remaining lectures:

- **Sampling** (L4 MC, L5 TD): estimates are formed from the trajectories visited rather than by sweeping all states, and require no knowledge of p.
- **Function approximation** (L8+): rather than one stored value per state, parameters $\mathbf{w}$ generalize across states.

Bridge to Lecture 4: dropping the model

DP's full update needs the dynamics:

$$v(s) \leftarrow \sum_a \pi(a \mid s) \underbrace{\sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v(s')\big]}_{\text{expectation over the } \textit{known } p}.$$

In most problems p is *unknown*. **Monte Carlo** (L4) replaces the expectation by an **average over sampled returns**:

$$V(S_t) \leftarrow V(S_t) + \alpha\big[\underbrace{G_t}_{\text{sampled return}} - V(S_t)\big]$$

— the $\text{New} \leftarrow \text{Old} + \alpha[\text{Target} - \text{Old}]$ skeleton from Lecture 1, now with Target = a return drawn by interacting.

Same GPI loop — evaluation changes from "compute the expectation" to "sample it". The named contrast: DP does a **full-width backup** (all successors, weighted by p); MC/TD use **sample backups** (TD: one sampled successor; MC: one sampled trajectory). This axis — *full vs sample* — and bootstrapping organize the next three lectures.

Summary — Lecture 3

- **From fixed point to iteration:** a Bellman equation $v = \mathcal{B}v$ becomes the update $v_{k+1} \leftarrow \mathcal{B}v_k$.
- **Two operators:** $\mathcal{B}_\pi$ (evaluation) and $\mathcal{B}_*$ (value iteration), both γ-contractions, so each has a unique fixed point (v_π, v_*), approached geometrically.
- **Policy-improvement theorem:** greedy w.r.t. v_π is never worse, and equality implies optimality; this is the basis of every value-based control method here.
- **PI vs VI** are the two endpoints of the spectrum of *how much evaluation between improvements*; **GPI** is the general loop instantiated by almost every method of the remaining lectures.
- The **curse of dimensionality** motivates the two later responses: sampling (L4–5) and function approximation (L8+).

Lecture 4 treats Monte Carlo methods: GPI when p is unknown, with the evaluation step done by *sampling complete returns*.